LAB QUESTION _6

Self-Balancing Binary Search Tree: Implementation and Comparison

Programme	MSc Artificial Intelligence & Machine Learning (MSc AIML)
Course	Data Structures and Algorithms Laboratory
Lab Topic	Implement a Self-Balancing Binary Search Tree and Compare
Duration	3 hours (lab session) + 1 week for the Self-Learning component
Maximum Marks	10 (Correctness & Demonstration 5 + Self-Learning 2 + Viva 3)
Language	Python / Java / C++ (student's choice; no built-in balanced-tree libraries)

1. Scenario

MediQueue is a telemedicine start-up that serves patients across several cities. Every new patient is given a numeric **Patient ID** at registration, and the platform must be able to **insert, search and delete** patient records within milliseconds, even when millions of records are stored.

The first version of the system indexed patient records using a plain Binary Search Tree (BST). It worked well during testing, where IDs were entered in random order. After launch, the team observed that search times kept growing steadily. An engineer found the cause: IDs are issued by an auto-increment counter, so records arrive in almost sorted order, and the BST degenerated into something that behaves like a linked list.

You have been hired as a junior data engineer. Your manager has asked you to **replace the index with a self-balancing BST (AVL Tree)** and to produce an evidence-based comparison that proves the new index is worth deploying.

2. Tasks

Part A: Baseline (Plain BST)

1. Implement a plain BST that stores a Patient ID (integer key) and a Patient Name (string value).
2. Support insert, search, delete and in-order traversal. Handle all three delete cases (leaf, one child, two children).
3. Add a function to compute the height of the tree and a counter for the number of key comparisons made.

Part B: Self-Balancing Tree (AVL Tree)

1. Implement an AVL tree with the same interface as Part A, storing the height of each node and computing its balance factor.
2. Implement all four rebalancing cases: Left-Left (right rotation), Right-Right (left rotation), Left-Right and Right-Left (double rotations).
3. Rebalancing must happen after both insertion and deletion. Count the total number of rotations performed.
4. Write an is_balanced() check that verifies every node has a balance factor in {-1, 0, +1}, and a BST-property check. Use them as assertions in your tests.

Part C: Workload Generation

Create the following three datasets for each size N = 1,000, 5,000 and 10,000 (use a fixed random seed so results are reproducible):

- **Dataset 1 (Sequential):** IDs 1, 2, 3, ..., N in ascending order (the MediQueue production pattern).
- **Dataset 2 (Random):** a random permutation of 1..N.
- **Dataset 3 (Nearly sorted):** ascending IDs with about 5% of the positions randomly swapped.

Part D: Experiments and Comparison

For each tree type and each dataset/size combination, record the following:

- Final height of the tree after all N insertions, and the theoretical minimum height (floor of log2 N).
- Total insertion time, and total rotations (AVL only).
- Average number of comparisons per successful search, measured over 1,000 randomly chosen existing IDs.
- Height of both trees after deleting a random 20% of the keys.

Present the results in a neat table (printed on the console or exported to CSV) and plot at least one graph (for example, N versus height for both trees on the sequential dataset).

Part E: Analysis (maximum one page)

Write a short conclusion that answers the following questions using your own measured numbers:

1. Which dataset produced the worst case for the plain BST, and what was its height and average search cost?
2. How much extra work (rotations, bookkeeping, memory for the height field) does the AVL tree pay for guaranteed balance?
3. On the random dataset, was the AVL tree noticeably better than the plain BST? Explain why or why not.
4. Would you recommend the AVL tree for MediQueue? Justify your recommendation.

3. Sample Input / Output

Insert the IDs 10, 20, 30, 40, 50, 25 in this order and print the tree structure and height.

Observation	Plain BST	AVL Tree
Root after all insertions	10	30
Height	5	3
Rotations performed	0	4 single rotations (two left rotations, then one right-left double rotation)
In-order traversal	10 20 25 30 40 50	10 20 25 30 40 50

Note: the in-order output is identical, which confirms that balancing never breaks the BST property.

4. Constraints and Instructions

- Do not use library classes such as TreeMap, TreeSet, sortedcontainers, std::set or std::map as the tree. Implement the node structure yourself.
- For the sequential dataset with N = 10,000, a recursive insert in the plain BST may exceed the recursion limit in some languages. Use an iterative insert for the plain BST or handle the limit explicitly, and mention which one you chose.
- The code must be modular (separate classes or files for BST, AVL, workload generator and experiment runner) and properly commented.
- Every result you report must come from your own program output. Do not copy numbers from the internet.

5. Deliverables

- Source code (zipped, or a GitHub link) with a README explaining how to run it.
- Output screenshot or CSV of the comparison table, and the graph(s).
- One-page analysis (Part E) and the Self-Learning note (Section 6).
- Live demonstration to the examiner, followed by the viva.

6. Self-Learning Component (2 Marks)

Using resources outside the classroom, study ONE of the following alternatives and extend your comparison:

- **Red-Black Tree:** how the colour rules approximate balance and why many libraries prefer it for insert/delete-heavy workloads.
- **Treap or Splay Tree:** randomised or self-adjusting balancing, and how it behaves on the sequential dataset.
- **AA Tree or Scapegoat Tree:** simplified or rebuild-based balancing.

Submit a note of at most one-page covering: the balancing idea, its worst-case height bound, how it differs from AVL, a small experiment (or working code) that supports your claims, and the references you used. Marks are awarded for the depth of understanding and the independent effort shown, not for the length.

7. Evaluation Rubric

Component	Criteria	Max	Awarded
Correctness & Demonstration	Plain BST works with all delete cases, height and comparison counter (1)	1	
	AVL tree with all four rotation cases, rebalancing on insert and delete, correct balance checks (2)	2	
	Workload generation, instrumentation and comparison table/graph produced from own run (1)	1	
	Live demonstration with the examiner's test input, plus clear explanation of the result (1)	1	
Subtotal: Correctness & Demonstration		**5**	
Self-Learning	Understanding of the chosen alternative tree and its height bound; clear comparison with AVL (1)	1	
	Independent effort: own experiment or code, references cited, insight beyond the lab sheet (1)	1	
Subtotal: Self-Learning		**2**	
Viva	Three questions from the viva bank, one mark each: concept (0.5 to 1), reasoning with own data (0.5 to 1), ability to answer follow-up (0 to 1 overall, adjusted by the examiner)	3	
Total		**10**	